Q09: How Does the Composition of Training Data Shape What a Model Knows and Can Do?

DLS 2026 — Module (Paradise)

October 2026

Learning objectives

After this module, a student should be able to (i) describe what web-scale pretraining corpora actually contain and how curation pipelines (extraction, filtering, deduplication) shape them, (ii) explain exact and MinHash-based fuzzy deduplication and why dedup improves training efficiency and reduces memorization, (iii) reason about quality filtering (heuristic, classifier-based, and perplexity-based) and its trade-offs, (iv) state what is known about repetition and reuse (the ∼4-epoch result), mixture weighting, curriculum ordering, and tokenizer–data interactions, and (v) articulate why data quality is now treated as a first-class lever alongside raw scale.

1 Introduction: from "more data" to "which data"

Scaling-law analysis made dataset size a headline number: compute-optimal training scales parameters and tokens together, with Chinchilla trained on 1.4T tokens rather than the ∼300B of its same-compute predecessor Gopher [2]. But two findings pushed the field past "just add tokens." First, extrapolating demand against the finite stock of public human text suggests the stock could be fully utilized between roughly 2026 and 2032 [11]. Second, controlled ablations show that *which* tokens you use changes benchmark performance by amounts comparable to modest scale increases: filtering the 15T-token FineWeb corpus down to 1.3T tokens of classifier-scored educational content (FineWeb-Edu) raised MMLU from about 33% to 37% and ARC from about 46% to 57% at a fixed 350B-token training budget [9]. Composition — sources, filters, duplicates, mixture weights, and ordering — is now a competitive discipline, not bookkeeping. This note treats each lever in turn, staying honest about which effects are established at what scale.

2 What is in a web-scale corpus?

Modern pretraining sets are dominated by Common Crawl snapshots, plus curated slices: code, Wikipedia, books, and papers. The Stack collects 3.1 TB of permissively licensed source code across 30 languages, and is the reference code corpus for open models [4]. Web crawl, by contrast, arrives as raw HTML of highly uneven quality; every major pipeline therefore performs text extraction, language identification, URL blocklists, heuristic quality filters, and deduplication before any training [9]. Domain composition matters in itself: pretraining 1.5B-parameter models on different domain mixtures shows that including heterogeneous sources (web plus books) is broadly beneficial, while single-source curricula are worse, and filter effects are not predictable from surface domain statistics

[6]. The upshot: a "token" is not a fungible unit. A token from a deduplicated Wikipedia article, a scraped boilerplate page, and a Python function carry very different learning signal per FLOP.

3 Deduplication

Exact and near-duplicate removal. Web corpora contain massive redundancy: one 61-word English sentence appeared over 60,000 times in C4, and removing near-duplicates and long repeated substrings let language models emit memorized text roughly ten times less frequently while reaching the same accuracy in fewer steps; deduplicated C4 is a few percent smaller and correspondingly cheaper per epoch [5]. Exact dedup (hashing documents or lines) is cheap but misses the common case of boilerplate templates and paraphrased copies.

MinHash and fuzzy dedup. Near-duplicate detection needs a similarity notion robust to small edits. The standard tool, MinHash, works on *shingles*: fixed-length token n-grams sampled from a document. If a document is a set S of shingles, the Jaccard similarity of two documents is $J(A, B) = |A \cap B|/|A \cup B|$. MinHash uses k random hash functions h_i and stores, per document, the signature $\big(\min_{s \in S} h_i(s)\big)_{i=1..k}$: the probability that two documents agree in one signature coordinate equals their Jaccard similarity. Signatures are bucketed via locality-sensitive hashing (banding) so only candidate pairs are compared, which is what makes this subquadratic at trillion-token scale. FineWeb ablated dedup granularity extensively; a notable result is that *global* MinHash dedup across all 96 snapshots removed ∼80% of data (from 20T to 4T tokens) and trained *worse* than per-snapshot dedup, which kept 20T tokens and matched RefinedWeb — aggressive dedup can discard useful variety [9]. Muennighoff et al. similarly report that dedup did not help downstream on their (already clean) C4-based setup, while noting it may still reduce memorization [8].

Why dedup helps when it helps. Three mechanisms: (1) *efficiency* — repeated documents waste gradient steps on already-seen content; (2) *memorization and privacy* — duplication is the strongest known driver of verbatim recall, so removing duplicates reduces regurgitation and extraction risk [5]; (3) *evaluation validity* — train/test overlap inflates benchmark scores. The caveat above is real: these benefits concentrate on noisy corpora, and overly aggressive dedup is harmful [9, 8].

4 Quality filtering

Three families of filters, in historical order:

1. **Heuristics:** C4-style rules (document length, ratio of punctuation/lines starting with bullets, presence of stopwords). Cheap and blunt; FineWeb's ablations found that keeping a small set of effective C4/MassiveText-style filters, tuned rather than piled on, was a good default [9].
2. **Perplexity filtering:** CCNet scores each document by the perplexity of an LM trained on a target domain (e.g., Wikipedia) and keeps low-perplexity text [12]. This is *domain targeting* disguised as quality: it selects for Wikipedia-likeness. Muennighoff et al. found perplexity filtering effective on noisy data but not on clean data [8].
3. **Classifier filtering:** FineWeb-Edu labels ∼460K sampled pages with Llama-3-70B-Instruct for educational value (0–5), trains a regressor on embeddings, and keeps pages above a threshold [9]. This "LLM-as-annotator" approach scales curation judgments, but inherits the judge model's biases: it optimizes *grade-school-educational* text, and the resulting 8× reduction in tokens is only a win because quality-per-token rose faster than count fell.

Filtering has a known dark side: aggressive quality or toxicity filtering trades benchmark performance against representation and safety properties, and no one-size-fits-all threshold exists; effects of filtering are not predictable from text statistics and must be ablated empirically [6]. The textbook-quality extreme — small, heavily curated, partly synthetic corpora — produced strong small models for code [1], demonstrating that quality can substitute for scale up to a point, with open questions about what such narrow data fails to teach.

5 Mixture weighting, repetition, and annealing

Domain mixtures. Training sets upsample high-value domains (Wikipedia, code, papers) far above their natural web frequency. DoReMi shows domain weights can be *optimized*: a small proxy model trained with group-DRO over domains yields weights that, applied to a 30× larger model, improved few-shot accuracy by 6.5 points over The Pile's default mixture and reached baseline accuracy in 2.6× fewer steps [13]. Mixture choices interact with the target: code in the mix has repeatedly been observed to benefit natural-language reasoning; Muennighoff et al. quantify one version of this, finding that filling up to half the data budget with Python code (from The Stack) gave roughly a 2× increase in *effective* tokens even when evaluating only natural-language tasks [8, 4]. The mechanism is hypothesized to be shared structure (long-range state tracking, compositional dependencies) rather than surface syntax, though we flag that code→math transfer specifically is reported anecdotally across models rather than established by a single controlled study.

Repetition: the ∼4-epoch rule. With 400+ runs up to 9B parameters, Muennighoff et al. find that with constrained data and fixed compute, up to about **4 epochs** of repeated data changes loss negligibly relative to unique data; beyond that, returns decay toward zero [8]. Their fitted law predicts that in the data-constrained regime extra compute should go disproportionately to epochs rather than parameters (smaller model, more passes) — unlike Chinchilla's equal scaling [2]. Caveats: their results are for repetition of a *whole* corpus from the start of training; up-sampling a single domain is a different intervention, and the resulting rule is a practical heuristic, not a constant of nature.

Annealing. A common recipe schedules the mixture over training: a large, diverse mix early, then a shorter "annealing" phase near the end (with decaying learning rate) on upsampled high-quality data. MiniCPM's two-stage recipe raised the mathematics and code share to ∼70% during the annealing phase of pretraining [3]. Reported across many models but rarely ablated in isolation at scale, annealing should be read as a converging practice with plausibly large effects and thin causal literature.

6 Curriculum and ordering

Beyond mixture schedules, the *order* of data matters at the margins. Data age is one documented axis: a temporal gap between pretraining and evaluation data degrades performance, and the gap is not repaired by finetuning [6]. Within a fixed mixture, uniform random shuffling is the default because it keeps the batch distribution stationary; easy-to-hard curricula can beat it for noisy data (RHO-LOSS selects learnable, relevant, not-yet-learned points, accelerating training on web-scraped data [7]), but results are setup-dependent and rarely survive the web-scale setting unchanged. The honest summary: ordering effects are real, modest, and mostly exploited via coarse mixture schedules (annealing) rather than fine-grained curricula.

7 Tokenizer and vocabulary interactions

The data mixture also flows into the *tokenizer*, which is trained on a sample of the corpus and fixes the tokenization for all of training. Consequences:

- **Fertility differences across domains:** code and non-English text split into more subword units per word than English prose, so a fixed token budget holds less code content; "tokens" as a unit of accounting are domain-relative. BPE-style subword tokenization [10] trades vocabulary size against sequence length, and its merge statistics reflect the tokenizer-training mixture.
- **Mixture accounting:** domain weights specified in tokens shift when measured in raw characters or bytes, since fertility differs by domain; a tokenizer trained on web text will fragment the scientific vocabulary of a papers-heavy mixture, spending capacity on rare subwords.
- **Vocabulary coverage:** a tokenizer trained without code (or without a language) makes code (or that language) artificially expensive, biasing the effective mixture away from what was declared.

A clean statement of the interaction is hard to find in the literature; the mechanism is uncontroversial but the magnitude of the effect relative to mixture weights is, to our knowledge, not separately quantified at scale — a good open question.

8 Why quality now rivals scale

Putting the pieces together: (1) the marginal value of fresh web tokens falls as the easy stock is consumed [11]; (2) repeated and code-augmented tokens remain useful, with quantified diminishing returns [8]; (3) classifier-based filtering shifts the quality-per-token distribution enough to change benchmark rankings at fixed compute [9]; (4) optimized mixtures buy training-step efficiency comparable to nontrivial compute increases [13]. None of these licenses the claim that "data quality beats scale" unconditionally — Chinchilla's compute-optimal frontier still binds [2] — but at fixed compute, composition is among the cheapest levers available, and it is the only lever whose gains compound with every downstream epoch.

What we did not establish. This note deliberately hedges: code→math transfer mechanisms, annealing ablations at scale, and tokenizer–mixture interaction magnitudes lack the controlled, multi-scale evidence base of scaling laws or the 4-epoch result. Treat those as strong practitioner priors, not settled science.

References

[1] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.

[2] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[3] Shengding Hu, Yuge Tu, Xu Han, et al. MiniCPM: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.

[4] Denis Kocetkov, Raymond Li, Loubna Ben allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The stack: 3 TB of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.

[5] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguists*, 2022.

[6] Shayne Longpre, Gregory Yauney, Emily Reif, Katherine Lee, Adam Roberts, Barret Zoph, Denny Zhou, Jason Wei, Kevin Robinson, David Mimno, and Daphne Ippolito. A pretrainer's guide to training data: Measuring the effects of data age, domain coverage, quality, & toxicity. *arXiv preprint arXiv:2305.13169*, 2023.

[7] Sören Mindermann, Jan M. Brauner, Muhammed T. Razzak, Mrinank Sharma, Andreas Kirsch, Winnie Xu, Benedikt Höltgen, Aidan N. Gomez, Adrien Morisot, Sebastian Farquhar, and Yarin Gal. Prioritized training on points that are learnable, worth learning, and not yet learnt. In *Proceedings of the 39th International Conference on Machine Learning*, 2022.

[8] Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin A. Raffel. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems*, 2023.

[9] Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.

[10] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016.

[11] Pablo Villalobos, Anson Ho, Jaime Sevilla, Tamay Besiroglu, Lennart Heim, and Marius Hobbhahn. Position: Will we run out of data? Limits of LLM scaling based on human-generated data. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[12] Guillaume Wenzek, Marie-Anne Lachaux, Alexis Conneau, Vishrav Chaudhary, Francisco Guzmán, Armand Joulin, and Edouard Grave. CCNet: Extracting high quality monolingual datasets from web crawl data. In *Proceedings of the 12th Language Resources and Evaluation Conference*, 2020.

[13] Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V. Le, Tengyu Ma, and Adams Wei Yu. DoReMi: Optimizing data mixtures speeds up language model pretraining. In *Advances in Neural Information Processing Systems*, 2023.